Compute Platform for AI

An Open-Source Summary

Thomas Debelle

KU LEUVEN

October 25, 2025

Contents

1 Lecture 1: Towards heterogeneous many-core processors 2
1.1 Scaling . . . 2
1.1.1 Denard broke down . . . 2
1.1.2 Dark and Dim Silicon . . . 3
1.2 Area for energy in single-core . . . 3
1.2.1 Super-scalar . . . 3
1.2.2 Memory area . . . 4
1.3 Area for energy through multi-core . . . 4
1.3.1 Homogenous . . . 4
1.3.2 Heterogeneous . . . 4
1.4 Area for energy through domain specific accelerators (DSA) . . . 4

2 Lecture 2: ISP's, GPU's and AI Workloads 5
2.1 ISP's & GPU's for graphics/images . . . 5
2.1.1 ISP applications, architecture and operation . . . 5
2.1.2 GPU applications, architecture and operation . . . 5
2.2 Deep learning algorithms . . . 6
2.2.1 What is deep learning? . . . 6
2.2.2 From neural nets to deep neural nets . . . 6
2.2.3 Classes of deep neural networks: . . . 6

3 Lecture 3: Efficient Deep Inference: Hardware Bottlenecks & Parallelization 8
3.1 Baseline and hardware challenges . . . 8
3.1.1 Metrics for execution efficiency . . . 9
3.1.2 A tale of two rooflines . . . 9
3.2 xPU processor enhancements for AI . . . 10
3.2.1 Parallelization (spatial unrolling optimization) . . . 10

4 Lecture 5: Efficient Deep Inference: Quantization 13
4.1 Block quantization . . . 14
4.1.1 PQT and QAT . . . 15
4.1.2 Mixed precision NN . . . 15
4.2 Variable precision `int` MAC's . . . 15
4.2.1 Data gating . . . 16
4.2.2 Multi-gating . . . 16
4.2.3 Add/shift-gating . . . 17
4.2.4 Bit-serial way . . . 17
4.2.5 Comparing . . . 17
4.2.6 From MAC to array level . . . 17
4.3 SoTA example of Quantization . . . 18
4.3.1 Envision (KU Leuven) . . . 18
4.3.2 Tensor Cores (Nvidia) . . . 18
4.3.3 Hybrid training / inference chip in 7nm (IBM) . . . 19
4.3.4 Binareye - digital and MS (KU Leuven & Stanford) . . . 19

4.4 In memory compute . . . 20
4.4.1 Concept . . . 20
4.4.2 Analog IMC . . . 20
4.4.3 Digital IMC . . . 21

5 Paper to Read **23**
5.1 Lecture 1 . . . 23
5.2 Lecture 2 . . . 23
5.3 Lecture 3 . . . 23
5.4 Lecture 5 . . . 23

6 Questions **23**
6.1 Lecture 1 . . . 23
6.2 Lecture 2 . . . 24
6.3 Lecture 3 . . . 25
6.4 Lecture 5 . . . 26

7 License **28**
7.1 Modification . . . 28

1 Lecture 1: Towards heterogeneous many-core processors

1.1 Scaling

In IC and chip design, there is two fondamental laws:

- **Denard's law:** as transistors get smaller, the power density remains constant, which leads to lower and lower supply voltage to avoid to break down the oxide due to a strong $\vec{E}$.
- **Moore's law:** every generation can fit twice as many transistors on a certain area.

1.1.1 Denard broke down

Denard's law is based on the fact that the width, length, oxide thickness and voltage is reduced by a factor α each time. This factor also influences:

- Density: α^2
- Capacitance: $1/\alpha$
- Delay: $1/\alpha$

Thus, Power = $CV_{DD}^2 f \propto 1/\alpha \cdot cst \cdot 1/\alpha = 1/\alpha^2$. Finally the power density is a constant.

On paper, this is valid but we can't infinitely scale down V or we will have lower speed the closer we come to V_{th} which is not feasible. Moreover, the wire cannot scale down as desired or we will have a bad resistance in the wire. This will make the wires a bit more "capacitive" and so the α factors will no longer cross out as Denard predicted. The power is constant and the power density is α^2 which is quite problematic.

1.1.2 Dark and Dim Silicon

The end of the Denard's scaling lead to a plateau in the power consumption of chips. We have to buy this energy efficiency. We know that the power density scales with α^2 at maximum clock frequency. So we will introduce:

- **Dim silicon:** silicon running at below max f_ϕ/α^2
- **Dark silicon:** $1/\alpha^2$ blocks that totally shut down when not used

In practice, if we scale with a factor $S = 2$ we can put 2^2 more silicon and the speed should increase by a factor 2. In *dark silicon*, we will still speedup the clock frequency but all the newly added cores will be shutdown. This is quite unsustainable as more cores (due to Adhalm's law) won't leverage from parallelism. The better idea is to not clock faster and use this extra factor 2 to produce dim silicon and then the rest with dark silicon.

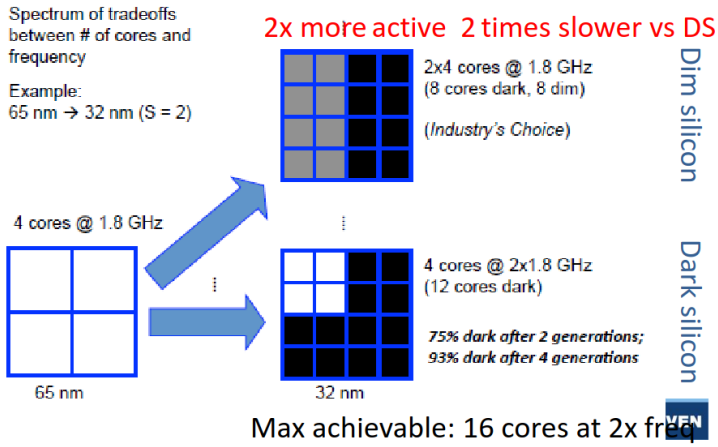

Figure 1: The two aforementioned techniques

Recently, we are using more and more dark silicon as accelerators that get turned on for specific task – accelerators.

1.2 Area for energy in single-core

To make a processor faster, we can use either:

- *Instruction-level parallelism*: VLIW, OOO super-scalar
- *Data-level parallelism*: SIMD, GPU

> Won't re-explain what those are, check the computer architecture lecture: *link*

Those are prime examples of dim silicon with lower clock speed for same throughput thanks to parallelism.

There is also the vector extension that can be seen as a sort of *temporal* data parallelism. SIMD, VLIW and other processing is often found in DSP chip since such processing is often data intensive.

1.2.1 Super-scalar

In this version, it is out of order and the processor can schedule instructions when it wants. It uses the Tomasulo's algorithm but must commit in order to avoid data dependency issues.

The advantage of such processor is the flexibility of the operations. Typically, some execution stage can take more time than others. We must use a reorder buffer (ROB) to commit in order. This is a prime example of spending extra transistors instead of clocking faster. There is a certain overhead for controlling such processors but the gains are far superior.

1.2.2 Memory area

Memory access time is the usual bottleneck in computing, a lot of time, the thread is blocked because it waits for data to arrive. Latency of cache and memory is quite important and to hide it we use multi-threading with some level of granularity to keep all cores as busy as possible.

We can't use too much of simultaneous multithreading SMT because the overhead is quite important. Need each time a program counter, register file and a reorder per thread. On top of that, the commit and scoreboard is more complex.

This is why we use various cache level and we exchange latency with transistors count. A recent trend is using large reorder buffer to see well in advance and to re-order online the instructions. This will increase parallelism and reduce off-chip memory accesses.

1.3 Area for energy through multi-core

Sadly, this is not enough and *Amdahl's law* explain that parallelism will not always lead to a speedup and lots of applications are not easily parallelizable.

1.3.1 Homogenous

We first started by doing `ctrl`+`c` and `ctrl`+`v` on the cores to realize a speedup. On top of that, we would some p-state (dim silicon) or shut it down completely (dark silicon) if not used.

1.3.2 Heterogeneous

Here, we will have dedicated low-power cores that can realize operations if low performance is needed. Depending on the workload, we can switch to efficiency core or performance core. This is present in the **big.LITTLE** ARM architecture. We must use the same instruction set to seamlessly transfer from one core to the other.

This idea is heavily used in embedded devices like smartphone and is starting to make its way through in personal computer, typically Apple's computer.

1.4 Area for energy through domain specific accelerators (DSA)

We are now facing the utilization wall. More and more silicon must be dim or dark, we must use the extra transistors for something. Here comes accelerators which are custom ASIC blocks for specific function.

2 Lecture 2: ISP's, GPU's and AI Workloads

2.1 ISP's & GPU's for graphics/images

2.1.1 ISP applications, architecture and operation

To take a picture, many steps are required: calibration, RGB conversion, encoding, post-processing, … Image Signal Processor is one of the earliest example of ASIC. The operations are fixed and easily paralleziable.

Initially, it was mostly hardwired but more recently introduction of more flexibility. It gained popularity in commercially available ISP as people demanded more from their smartphones and cameras.

2.1.1.1 IPU This is becoming more and more like an Image Processing Unit where we want the best of both world:

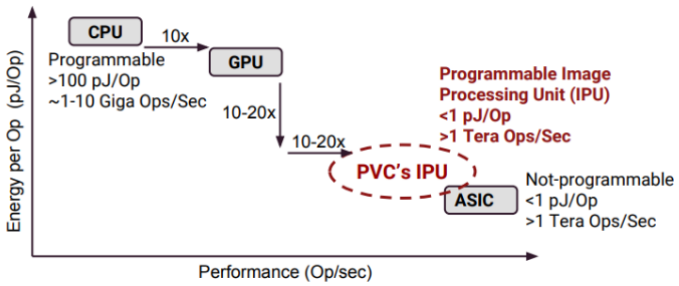

Figure 2: IPU

This is the idea introduced in the google pixel family with their Pixel Visual Core that allows data processing alongside the CPU. They use multiple small ASIP and use their locality on the die to exchange information between each other. It is a fast paced processing where data is consumed and sent to the next processor.

They arange multiple Processing Elements (PE) next to each other and hardwired them all together. They are composed of ALU's and MAC units and can be programmed and sent to the next PE. The dawn of NPU…

In short, the goal is to exploit **parallelism** and functional pipelining. We try to provide more programmability without losing in efficiency.

2.1.2 GPU applications, architecture and operation

> Repeat of Computer Architecture, Summary here.

2.1.2.1 Mobile Here we will try to maximize throughput under power and area limitation. We will also use extensively lower precision data types that can relieve stress and increase throughput while minimizing its impact on the output quality.

We will more and more see tight integration with the CPU and GPU with shared coherent memory space. More and more programmability with CUDA or openCL. Anything that can be parallelized is interesting to run on a GPU.

2.2 Deep learning algorithms

> **Definition of AI**
>
> Any program that mimics human intelligence. A program that can sense, reason, act and adapt

2.2.1 What is deep learning?

- **Machine learning**: is a subset of AI and it is an *algorithm that is able to learn from data*.
  - **Deep learning**: a representational machine-learning using multi-layered neural networks (NN)

The idea of a deep-learning network is one that has many layers and the computer truly learns on its own without much human interaction or tuning. It learns its own representation of the world or task he must accomplish.

2.2.2 From neural nets to deep neural nets

A basic neural network is composed of many **nodes** which forms a **layer**. They are often *fully-connected* to the next layer. The NN will adjust its weight based on algorithm during the training phase. Weight represents how much the information of one node will be forwarded to the next one. The AI can also tweak the **bias** to adjust its performance. Finally, to reduce the "*fuzziness*" we use some **activation functions**.

$$o_n^l = \sigma\left(\sum_m w_{m,n}^l \cdot o_m^{l-1} + b_n^l\right)$$

Table 1: Various σ activation function

Name	Function	Good in math	Easy in HW
Sigmoid	$\frac{1}{1+e^{-x}}$	**V**	**X**
Tanh	$tanh(x)$	**V**	**X**
ReLU	$max(0, x)$	**X**	**V**
Leaky ReLU	$max(0.1x, x)$	**X**	**V**
Maxout	$max(w_1^T x + b_1, w_2^T x + b_2)$	**X**	**V**
ELU	$x \geqslant 0; x$ else $\alpha(e^x - 1)$	**X**	**V**

By using labeled data, we can train the AI to accomplish a task. Using the AI to do for example pattern recognition is called *inference*.

2.2.3 Classes of deep neural networks:

2.2.3.1 DNN In the simple example of NN, we had a vector as an input. But, we can also have an image (matrix) as the input which we can apply the same pattern. This time, we will use a matrix notation to realize 2D-2D operations; a fully connected layer looks like:

$$o^l_{nx,ny} = \sigma\left(\sum_{mx,my}^{M,M} w^l_{mx,my,nx,ny} \cdot o^{l-1}_{mx,my} + b^l_n\right)$$

The matrices are quite large and this can be detrimental sometimes as one result is based on the full image.

2.2.3.2 CNN We are no longer fully connected, it is a sparse matrix. It has better convergence and faster training.

$$o^l_{nx,ny} = \sigma\left(\sum_{fx,fy}^{FX,FY} w^l_{kx,ky} \cdot o^{l-1}_{xn+kx,ny+ky} + b^l_n\right)$$

This will avoid to have a M matrix but rather a small kernel that is sliding over the matrix and of size $Fx \cdot Fy$. We can go further and perform some tensor kernel to apply the same or different kernel on multiple inputs. Each inputs can influence the same output. A kernel is of size $Fx \cdot Fy \cdot C$.

$$o^l_{nx,ny,k} = \sigma\left(\sum_{fx,fy,c}^{FX,FY,C} w^l_{kx,ky,c} \cdot o^{l-1}_{xn+kx,ny+ky,c} + b^l_n\right)$$

This operation requires 7 nested for loops. Bad news for software engineer, good news for hardware engineer has it opens the door for massive parallelism.

To avoid the size to get too big, we often finish by a **Pool and normalization** layer to "compress" the info. Finally we inject it in a classic fully connected NN to output a scalar result.

There are many architectures and flavor of CNN, each with their pros and cons, trying to solve different problems.

DO THE DEPTHWISE POINTWISE CONV EXPLANATION

2.2.3.3 Transformer and LLM The secret sauce of current LLM's is the use of the **attention** mechanism. It is an algorithmic representation of linguistic property of words and sentences. It will connect words between each other regarding the context which allows to not only process one word but also its context surrounding it. The maximum path length is only $\mathcal{O}(n)$ as the information is already embedded in the word after the attention process.

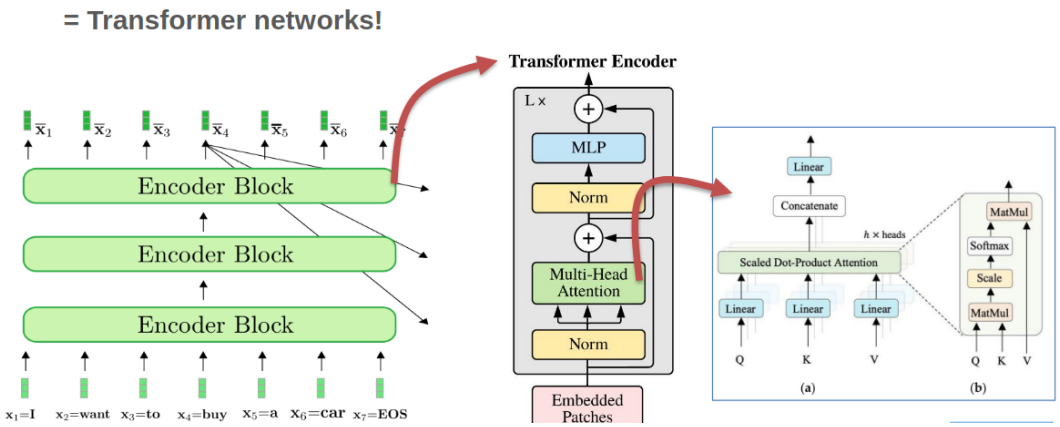

Figure 3: Transformers encoder

The query is for one word we want to inspect, the keys are the result of the query. This form a linear transformation which will tweak the high-dimensional embedding space to “distort” and bring words that are alike closer together. Finally we use the value matrix that will realize a linear transformation to help and find the next most probable word.

$$softmax(\frac{\underline{x}Q_w * XK_w^T}{\sqrt{100}}) * XV_w$$

Multiplication can be quite annoying but with efficient hardware, it can be accelerated. The real bummer here is the $softmax$ function:

$$softmax = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_i}}$$

Where x_i are values. The issue is that it can only be computed after processing all of the data. It is also a non-linear function that cannot be realized in one single pass. This means that we must re-access value from the cache which is slow and **not** energy efficient. Same story for the $norm = \frac{value_{original} - \mu}{\sigma}$.

After doing this pre-fill and encoding, we want to generate some new tokens. This is handled by the decoder block.

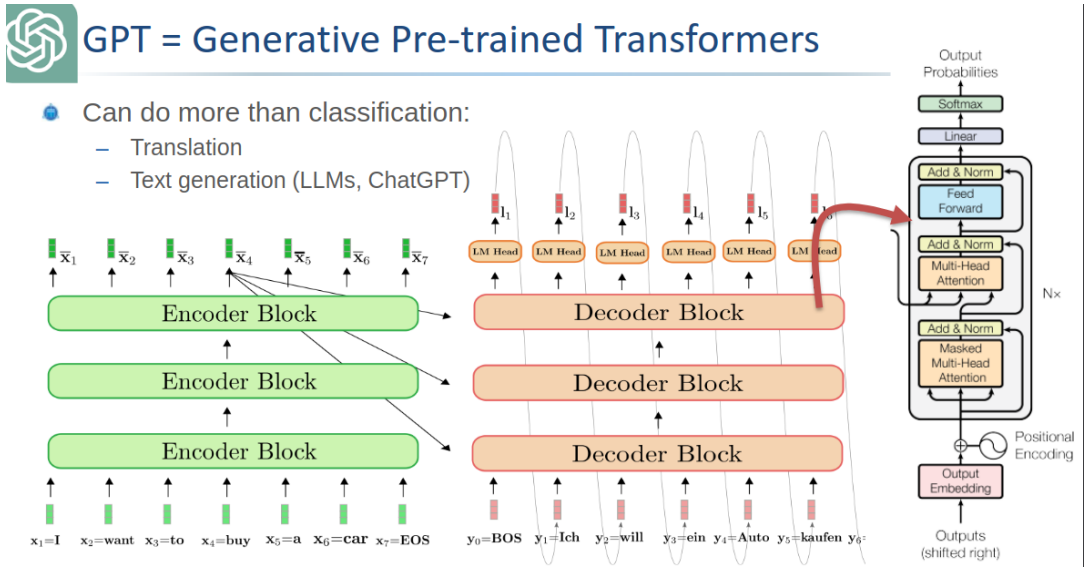

Figure 4: Full architecture

Those matrix multiplications are quite repetitive, we can thus save the result in what we call **KV cache**. We only append the new token and perform some vector matrix computation instead of the full matrix matrix computation.

- Prefill: compute limited
- Decode: memory limited

On a side note, most LLM’s are now *decoder-only* but we still have this prefill stage.

3 Lecture 3: Efficient Deep Inference: Hardware Bottlenecks & Parallelization

3.1 Baseline and hardware challenges

The two main efficiency metrics are:

- Latency: time per inference, time to get the first token
- Throughput: inferences per second

We often represent operations per second as TOPs - 10^{12} operations per second. When batching (running multiple queries at once) we gain in efficiency and it provides better throughput as we can reuse the weight of layers loaded in RAM for other users at the same time.

We also look at the energy and power. Where, TOPs/W (or TOP/J) matters depending of the scenario (cooling, embedded, infrastructure, … constrained)

With Moore's law, we should get twice the efficiency roughly every 2 years. The issue is that AI's complexity is getting more and more complex at a higher rate leading to a massive gap between computation abilities and needs.

We can fight at multiple font:

- TP/Latency: $time/op \cdot 1/utilization \cdot ops/inf$
- Energy: $energy/op \cdot 1/utilization \cdot ops/inf$

3.1.1 Metrics for execution efficiency

Matrix multiplications are needed in prefill stage. We call this operation a General Matrix Multiplication or GeMM. This type of optimized operations are implemented in the BLAS library that ensure fast and smart computation. A CPU has small **Processing Element (PE)** with specific datapath to accelerate such calculations.

The main drawback is the recurrent movements of data from on-chip to off-chip. A typical NPU will try to optimize those steps. Sadly, we will hit the memory wall which will lead to unavoidable latency and energy constraints.

One solution is to use lower precision operations to reduce the energy cost. Thankfully, energy savings is linear with the precision but the quality of the LLM isn't, especially if we use smart algorithm.

3.1.2 A tale of two rooflines

The roofline diagram we are familiar with is the $AI - TOPs$ one. Where we plot the Arithmetic Intensity as Operations per bytes of memory access. We have one horizontal line where there is the maximum compute in TOPs N_{op} and a diagonal line that is $AI \cdot BW$ depicting the bottleneck of the memory access. The ideal operating point is where those two line crosses as we are using the maximum of memory and compute. Otherwise the maximum attainable performance is $min(AI \cdot BW, N_{op})$.

But we can also have the **energy roofline**. We wil have the horizontal line being the minimum for an operation (without the transfer and other operations) and the the energy per DRAM access that is AI/E_{DRAM} access. We have the attainable efficiency as

$$\text{Attainable efficiency} = \frac{1}{E_{comp} + E_{mem}/AI}$$

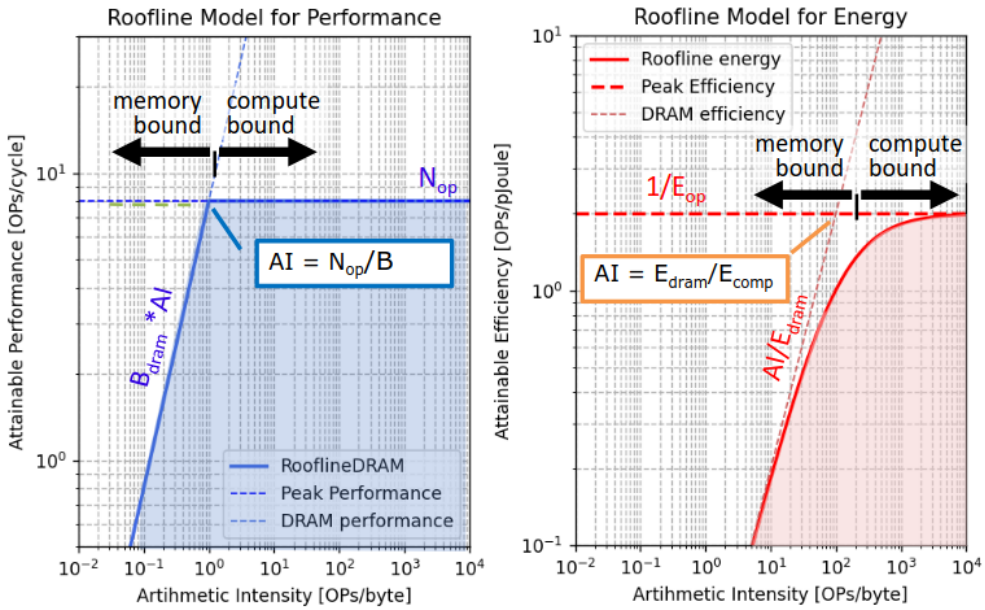

Figure 5: The rooflines diagram

We can see that the optimal performance point may not be the most energy efficient one! It all depends on our goals and seeing the large scale AI infrastructure we prefer to be slightly compute bound to avoid excess energy overhead.

3.2 xPU processor enhancements for AI

As explained earlier, the extra transistors we are gaining thanks to Moore's law need to be used to something. That's why we choose to do specific ASICS block or xPU. They all rely on one of the 5 tricks explained in the coming subsections.

Every trick here will try to push some limits (AI, BW, ...) to further improve the performances.

3.2.1 Parallelization (spatial unrolling optimization)

3.2.1.1 Parallelization in CPU and GPU cores Sadly, simply parallelizing work won't reduce the AI or increase it. We need to also push the peak performance or else no gain.

This is why we are interested in SIMD (parallel MAC in FP), Super-scalar (parallel load/store and compute) and also multi-core parallelism.

A good solution that is often employed is **fused multiply add** to avoid the extra load/store. Multiple inputs for one output, we almost divide memory access by 2.

Basic GPU's do not have this fused MAC in the classic CUDA cores, only massive parallelized MAC.

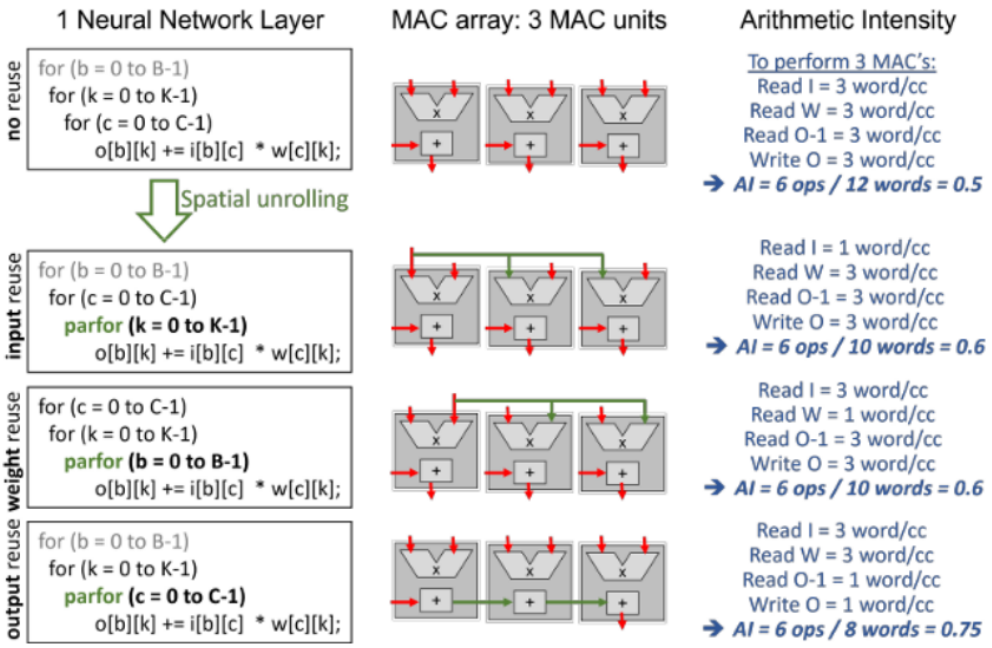

Figure 6: The goal is to reuse data to reduce AI

3.2.1.2 Advanced spatial data reuse in NPU's & Tensor Cores Of course, if we could unroll and reuse everything it would be ideal but the values are getting quickly out of hand. For example, with $B = 9; C = 4; K = 8$ where $B \times C$ and $C \times K$ are the inputs and weights dimensions respectfully. We thus need $B \cdot C \cdot K = 288$ MAC to do all the computations all at once. For the amount of access we have $K \cdot C + B \cdot C + B \cdot K = 8 \cdot 4 + 9 \cdot 4 + 9 \cdot 8 =$.

But we can't have such high number of mac even for those simple architectures. We often have only $100 - 1000$'s MAC in an accelerator. We will smartly fold convolutions on multiplier array. The main optimization criterion will be to optimize towards minimal memory access.

A good way is to mix spatial and temporal unrolling. For example do sub-vector sub-vector multiplication. The compiler can also optimize the loops and re-organize (tiling, ...), this is **dataflow optimization**.

```
for (b = 0 to B-1); // for each image in the batch
  for (k = 0 to K-1); // for each output channel
    for (c = 0 to C-1); // for each input channel
      o[b][k] += i[b][c] * w[c][k];

// Compiler optimization - tiling + spatial optimization
for (b2 = 0 to B/S-1); // for each image in the batch
  for (k = 0 to K-1); // for each output channel
    for (c = 0 to C-1); // for each input channel
      parfor (b1 = 0 to S-1); // for each image in the batch
        o[b2*S+b1][k] += i[b2*S+b1][c] * w[c][k];
```

In this code example, we are dong *weight reuse* as we the weight are reused (the indices are not impacted by a parfor loop so only 1 will be loaded at a time). If we insert the parfor in any other lines we will do some input or output reuse.

Table 2: Comparison of techniques, S = spatial reuse factor

	Weight reuse	Input reuse	Output reuse → FMA
Weight BW	1	S	S
Input BW	S	1	S
Output BW	$S+S$	$S+S$	$1+1$
AI	$2S/(3S+1)$	$2S/(3S+1)$	$2S/(2S+2)$

3.2.1.3 State of the Art examples

The real magic sauce of NVIDIA is their tensor cores. The major speed up comes from complex instructions such as HMMA and IMMA (Matrix Multiply Accumulate). It allows us to work with tensor and realize such operation efficiently. According to NVIDIA, such operation only has a $16-22\%$ of overheard compared with scalar or vector mac which can have up to 2000% of overhead.

Most of tensor cores in a NVIDIA chip are quite "small" they realize operation on 4x4 tiles to increase utilization.

For a 4x4 we have 128 operations and 16 input, weight access and 16 output read and access totaling at 64 memory access. The arithmetic intensity is $128/64 = 2$. Of course, we have to do tiling for bigger matrices, the code can look like:

```
for (b2 = 0 to B/PB-1);
  for (k2 = 0 to K/PK-1);
    for (c2 = 0 to C/PC-1);
      parfor (c1 = 0 to PC-1);
      parfor (k1 = 0 to PK-1);
      parfor (b1 = 0 to PB-1);
        o[b2*PB+b1][k2*PK+k1] += i[b2*P+b1][c]*w[c][k2*PK+k1];
```

Tensor cores give a 10x boost to operations. Other improvements are such as larger kernel, support for other data type. Flexibility and shared architecture for FP16 and INT4.

3.2.1.4 Tesla NPU

Instead of a 3D approach, they do a classic GeMM operation:

```
for (c = 0 to C-1);
  parfor (k = 0 to 95);
  parfor (b = 0 to 95);
    o[b][k] += i[b][c] * w[c] [k];
```

We must have a large bandwidth to process all of this together. This is quite large and only a vertical company can choose this solution. Since NVIDIA must support multiple form of workload, they cannot bet that everyone will use 96x96 matrix multiplication but Tesla can make this and reduce underutilization.

3.2.1.5 Huawei DaVinci

```
for (b1 = 0 to B/16-1); //for each image/pixel in the batch
  for (k1 = 0 to K/16-1); //for each output channel
    for (c1 = 0 to C/16-1); //for each input channel
      parfor (b2 = 0 to 15); //for each image in the batch
      parfor (k2 = 0 to 15); //for each output channel
      parfor (c2 = 0 to 15); //for each input channel
        o[b][k]+= i[b][c] * w[k][c];
```

But which is better for 1024 MAC's ? 2D or 3D ?

- 2D: we will have to re-access 32x32 output making the total access cost $32 + 32 + 2 * 32 * 32$ For $2 * 1024$ which results in a total AI of $2048/(2 * 32 + 2 * 1024) \approx 1$.
- 3D: we will have 8x8x16 MACs with $2(8 * 16) + 2 * (8 * 8)$ memory access for the same amount of operations leading to $2048/(2 * 128 + 2 * 64) \approx 5.4$.

In this case 3D is much better, but is it always ?

3.2.1.6 Multi-core parallelism Use the `parfor` loops at a higher level not just the datapath! We must communicate between core and split data among core until gathering all the results. It is **sharding**.

```
parfor (b = 0 to B-1); // Unrolling at the multi-core level
  for (k = 0 to K-1);
    parfor (c = 0 to C-1);
    o[b][k] += i[b][c] * w[c][k]; // Spatial unrolling at the core level
```

Table 3: Distributing and parallelizing

Options	Input	Weight	Output	Comment
Data parallelism	User data split on different GPU's	Replicated	No gathering needed	Everything is working in parallel no need to gather at the end
Tensor parallelism	Replicate the input across	Distributed (row-wise)	Gathering	We slice the tensor and each GPU's realize part of the MMA and then recombine all
Tensor parallelism	Replicate the input across	Distributed (column-wise)	Gathering	Like tensor parallelism but in the other dimension
Pipeline parallelism	All input go to the same GPU's	Set of weight per GPU's	Output are grouped together (unicast)	We distribute the weights across GPU's, can lead to underutilization as not every layer needs this much operation compared to other

There is not just one perfect technique, most of the time, we must combine different techniques depending on our need, architecture, FoM, ...

SambaNova startup is also believing in parallelism with intelligent on chip data routing for efficient distribution of computations.

At the end, we all try to raise the roof and tend to higher arithmetic intensity to gain in computing power.

4 Lecture 5: Efficient Deep Inference: Quantization

Using quantization will result into better roof as we can push the throughput further with low-accuracy operation using the same hardware. Instead of using full precision FP32, we can use lower bit precision as 4-bit, 1-bit, ... Lower order precision is often computed in **`int`**

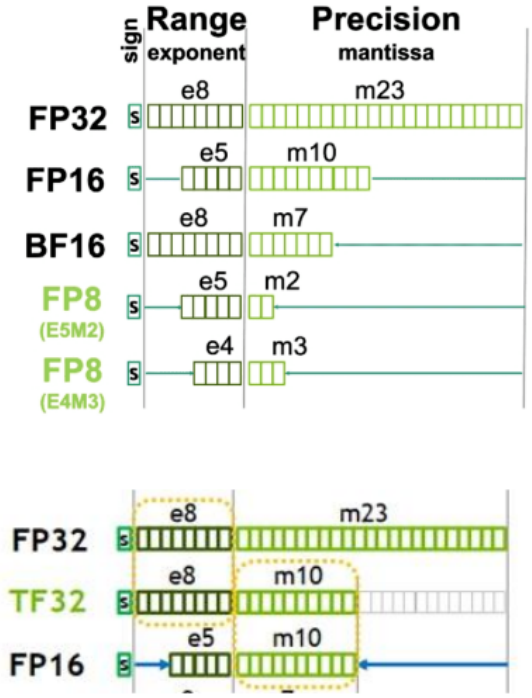

Figure 7: Datatypes for quantization

The TF32 format is used inside Nvidia's card in memory. It is not a real 32 bit float. They use this format as doing fp32 fp32 operation will unavoidably lead to trimming and rounding errors.

4.1 Block quantization

We can group exponent together and then use the mantissa for the operation. It is a form of **dynamic** fixed point arithmetic. The exponent is fixed and the mantissa can still vary. This allow for smart operation that can re-use previously computed results.

We can push this idea further by trying to group more than just the same components together. We can group multiple numbers under the same scale. This is the basic idea behind int quantization:

$$w_{quant} = quant(S_q \cdot (w - O_q))$$

We must add an offset O_q in the case the distribution is not symmetric.

Table 4: The MX-compliant data types

Format Name	Block Size	Scale Data	Scale bits	Element data format	Element bit-width
MXFP8	32	E8MO	8	FP8 (E4M3 / E5M2)	8
MXFP	32	E8MO	8	FP (E2M3 / E3M2)	6
MXFP4	32	E8MO	8	FP4 (E2M1)	4

Format Name	Block Size	Scale Data	Scale bits	Element data format	Element bit-width
MXINT	32	E8MO	8	INT8	8

4.1.1 PQT and QAT

The cheapest way to implement quantization is to do some **Post-training quantization**. After training, an expensive and time consuming step, we apply the quantization on the weights. We need some calibration data to make sure the quantization will not impact negatively the model.

On the other hand, **Quantization aware training** is ran at training. We quantize on the fly and and finetune with the training data. This is often better but more costly especially with LLM's.

Figure 8: PQT vs QAT

4.1.2 Mixed precision NN

In the same idea as quantization per group, we can apply different quantization order in the neural network. During the training, all possibilities exist and then model will adapt the π transfer function. Finally certain layers will prefer different quantization order than other.

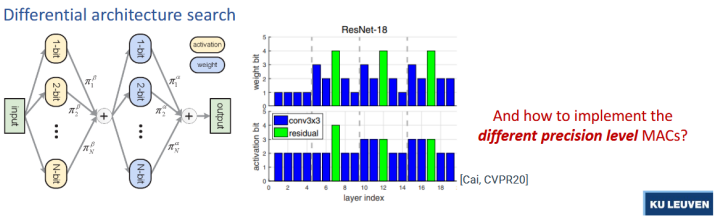

Figure 9: Differential architecture search

4.2 Variable precision `int` MAC's

All of this is cool and interesting but if we can't reuse hardware or do something a bit smarter, we cannot leverage from those algorithmic technique in real life. This is where versatile MAC arrays come into play.

4.2.1 Data gating

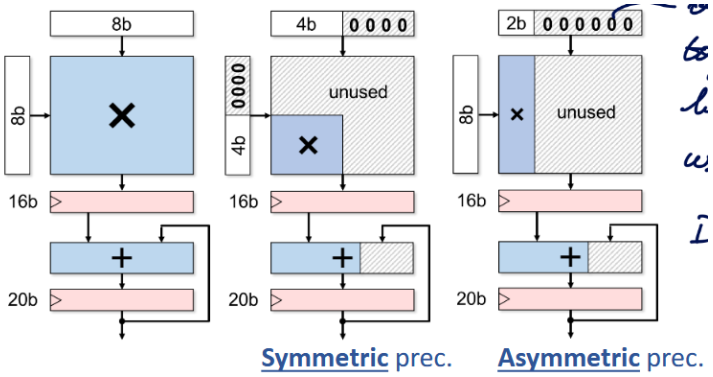

Figure 10: Data gating MAC

It is a relatively naive idea and will let lots of space unused ! We gate the LSB's to avoid unnecessary toggling and reduce energy consumption.

4.2.2 Multi-gating

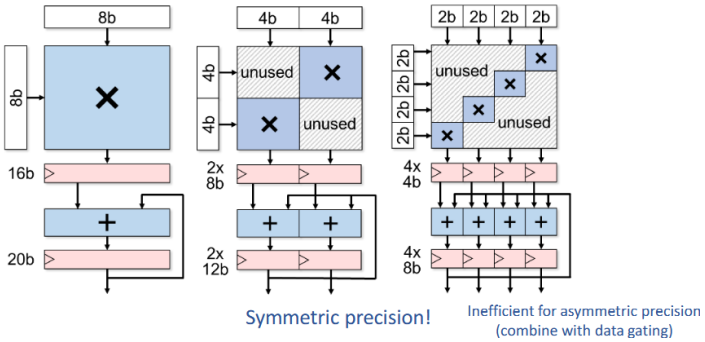

Figure 11: Multi-gating MAC

This can only be done for **symmetric** operations as asymmetric is quite inefficient. Interestingly, the more sub-word we have, the larger are the registers getting. It is not depicted in the figures, but we must also ensure some gating of the signal to avoid carry propagation between each sub word.

4.2.3 Add/shift-gating

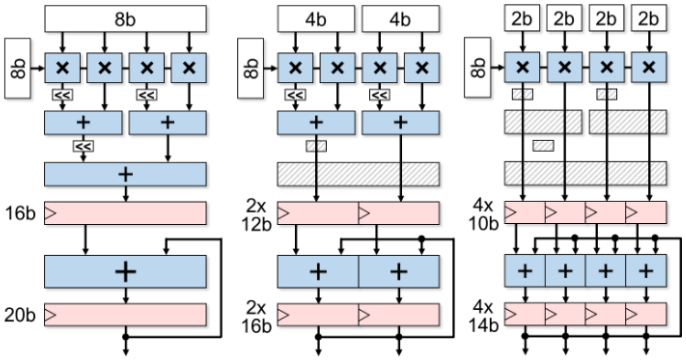

Figure 12: Add/shift-gating MAC

This method is suitable for asymmetric operations. We operate on sub-unit and combine the results as required. This method presents a finite granularity and the more range we want to support the more adder need to be chained hurting the critical path.

4.2.4 Bit-serial way

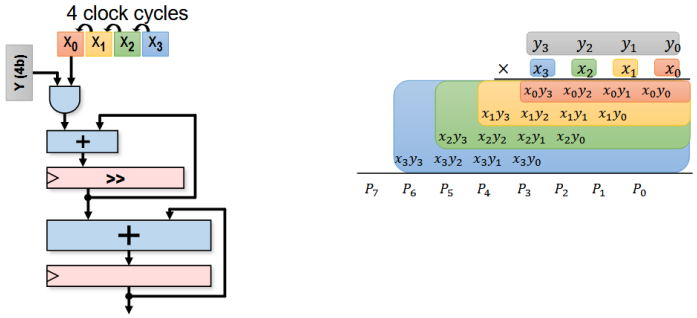

Figure 13: Bit-serial way MAC

We chain operations depending on the width and adjust the clock cycle.

4.2.5 Comparing

At full precision, data-gating is the best as there is little to no extra hardware or register toggling. But if we look at a reduced precision bit-serial and add-shift have the best energy usage and the best hardware usage.

4.2.6 From MAC to array level

For fair comparaison, we should compare at the MAC array as it presents many data sharing opportunities.

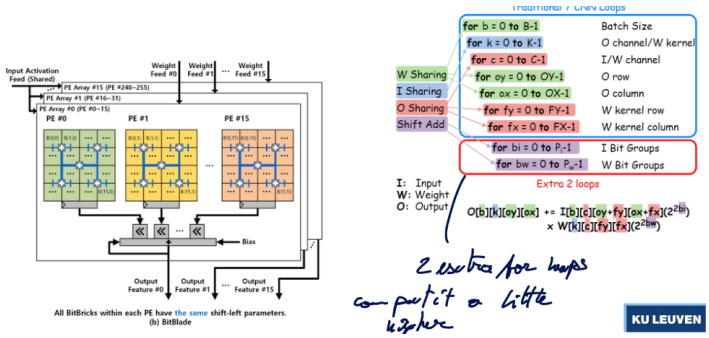

Figure 14: Data sharing can be seen as 2 extra for loops

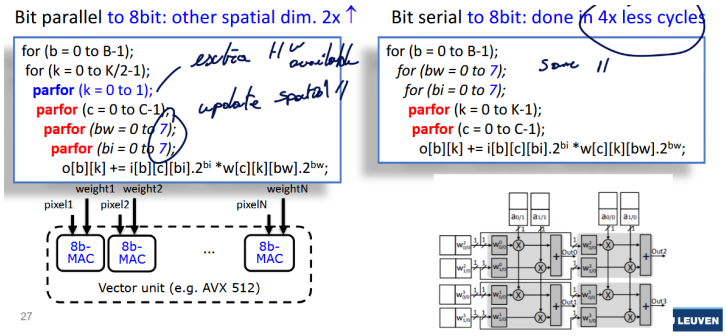

Figure 15: Potential gains of sharing

4.3 SoTA example of Quantization

4.3.1 Envision (KU Leuven)

It is a mult-gating approach with precision scalability. There are 256 MACS in a 16 by 16 array.

```
for (c = 0 to C-1);
  parfor (k = 0 to 15);
    parfor (b = 0 to 15);
      o[b][k] = i[b][c]*w[c][k]

// Can also be acting 512 MAC units and so on...
for (c = 0 to C-1);
  parfor (k = 0 to 15);
    parfor (b = 0 to 31);
      parfor (bw = 0 to 7);
        parfor (bi = 0 to 7);
```

This was successfully taped out and tested. They managed to adapt the supply voltage to gain even more in power efficiency.

4.3.2 Tensor Cores (Nvidia)

Same ideas occurs here where we can sort of extend the tensor in multiple dimensions if operating different datatype.

4.3.3 Hybrid training / inference chip in 7nm (IBM)

IBM has been pushing for lower and lower order of quantization. Proposing special structure for LLM inferences. They have some multi-precision compute array MPE with 16-way hybrid FP8 and 8-way FP16 engine in it. They do the same for int4 and int2 but they separate fp and int as they witnessed a significant energy saving and slight area reduction.

4.3.4 Binareye - digital and MS (KU Leuven & Stanford)

This extreme 1-bit quantization where we can use XOR gate to compute the multiplication. This is highly efficient in area and energy. The main drawback is the gathering of all those results which constitute the main bottleneck of this architecture.

The results were quite impressive for such low-order operations. This opens the possibility to run lightweight model on very efficient hardware.

We have really good weight stationary and we keep feeding input in parallel. We break down the input into Fx*Fy and repeat the feeding process for x*y cycles.

```
for (k2 = 0 to K/64-1); //for each output channel
  for (x = 0 to X-1); //for each in/output column
    for (y = 0 to Y-1); //for each in/output row
      parfor (c = 0 to 255); //for each input channel
      parfor (fx = 0 to 2) ; //for each kernel row
      parfor (fy = 0 to 2) ; //for each kernel column
      parfor (k1 = 0 to 63); //for each output channel
      o[b][k1+64.k2][x][y]+= i[b][c][x+fx][y+fy]
      * w[k1+64.k2][c][fx][fy];
```

4.3.4.1 Reducing the gathering bottleneck We can replace those large neurons array by a Mixed Signal approach that is composed of a switched-cap adder (sorts of ADC).

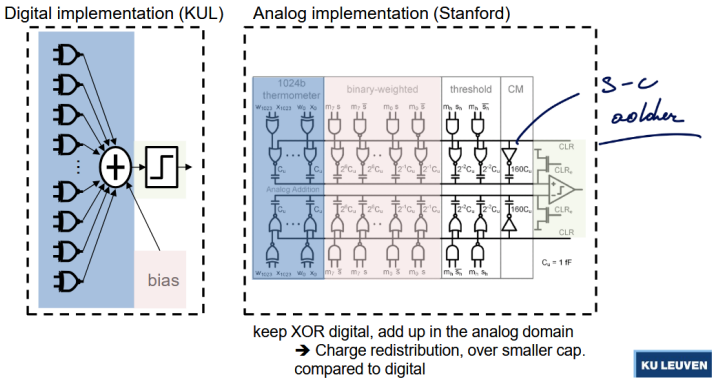

Figure 16: The switched cap implementation

This presented multiple issues:

- Variations of cap: using cap is better than a resistor ladder, … but high variation could make accuracy plummet
- Noise: again sufficient margin must be taken and extra technique should be employed to reduce impact of the noise
- Comparator offset: this can be quite detrimental but at least we can calibrate this effect reducing its impact

Even with this, the results are still stochastic around the perfect digital model. The energy used by the neuron array is reduced by a factor 12.

4.4 In memory compute

4.4.1 Concept

Instead of transferring all the time the data from memory, bring the computation in memory. The idea is to use the bitcell and use the selection line as the input and the weight are saved in the bitcell (weight stationary). So the data lines are actually the output lines.

By measuring the amount of discharge, we could be able to compute the result and digitize it.

4.4.2 Analog IMC

This seems like a good idea on paper but the amount of extra hardware around is important. We need to use DAC to convert the inputs in analog domain, sum up the current and digitize it with ADC. Finally some post process can be applied (ReLu, …)

Typically spatially unroll C*FX*FY in vertical direction. Unroll K in horizontal direction, B,X,Y temporal

4.4.2.1 Input pulse width Use bit width pulses to discharge a certain amount each time.

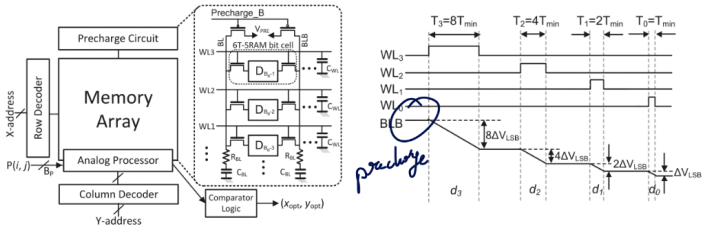

Figure 17: bit width pulses

We have superposition of binary weighted bitline discharges across multiple rows. Essentially a MAC operation with digital inputs and analog output. The inputs are now digital.

Quite nonlinear & random, difficult to manage large mismatches between SRAM cells

4.4.2.2 Input pulse count Here, all the pulses are the same, the only difference is the amount of pulse. Current-based addition still present large bitline non-linearity. This can be adapted using some non-linear modeling to counter-act it.

Both techniques use memory element as *current-source like* and the summation is realized with precharge-discharge cycle.

4.4.2.3 Voltage We use a regular resistance with ReRAM (or Flash) cell. We make resistive weighted sum.

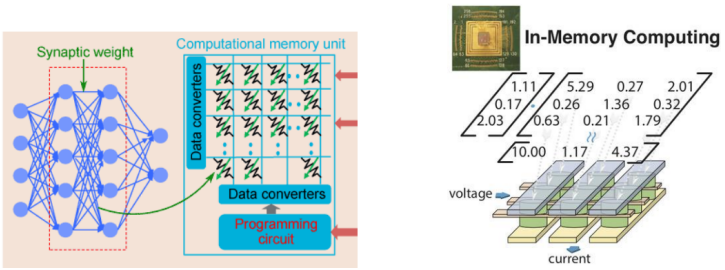

Figure 18: Voltage based IMC

This is kinda like the PE unit of google where we could save full the weight and directly route on the chip the data to go through multiple layers.

- THE GOODS:
  - Analog in-memory compute allows extreme input and output reuse
    - Due to compact memory (multiplier cell & added) & large arrays
  - Analog in-memory compute enables extreme weight stationarity
    - Program weights once, and leave them there forever?
    - If weight reloading is rare (needs large arrays, or small networks…)
      - Throughput/area increase (dense computing)
      - The larger is the memory the higher is the benefit. But not every NN will be this large to actually witness any gains.
- THE BADS:
  1. Can all NN layers exploit large arrays? → underutilization!
  2. Only precise at low quantization level? → Chip-specific training? Accuracy loss?

4.4.3 Digital IMC

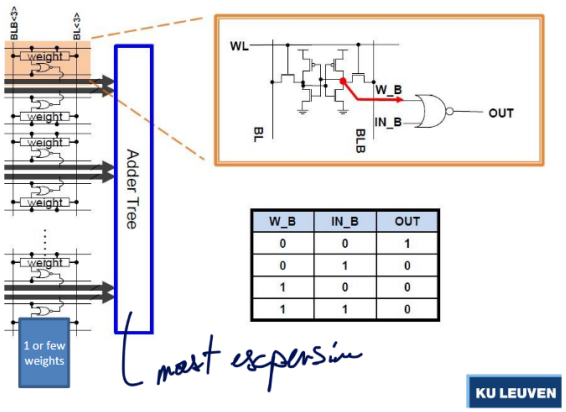

Figure 19: Digital IMC architecture

We digitize the 1*1 bit results and accumulation is done digitally. This adder tree is now the bottleneck. 1 weight per adder or weight bank. Here, results are deterministic which is better suited for the industry.

For example, TSMC provides special node for this kind of IMC. They share multiplier across 4 weights, it's not really IMC in the strict sense.

More freedom:

- Type of memory cell (now SRAM)
- Number of bitcells (weights) per multiplier $\rightarrow$ local data reuse (higher level stationarity)
- Number of cells per core (=size adder tree), or reconfigurable?
- Number of cores
- Data sharing between cores

Benefits of analog in memory compute still hold (but less!) - In-memory compute allows extreme input and output parallelism (reuse) - In-memory compute enables extreme weight stationarity

Compared to analog: - More flexible (reconfiguration of data reuse in function of layer topology?) - More reliable (precision guaranteed) - But… less dense (Tops/mm2 lower) - But… less efficient (Tops/Watt lower (at core level, not at system level?))

5 Paper to Read

5.1 Lecture 1

> **Paper to read**
>
> *Paper1*: A New Golden Age for Computer Architecture J. Hennessy AND D. Patterson
>
> *Paper2*: Apple M1: Ditching x86 A. Frumusanu
>
> *Paper3*: Paper2: Apple M1 deep dive: Micro-architecture (pg2) Anandtech, Andrei Frumusanu; *link related to the article*

5.2 Lecture 2

> **Paper to read**
>
> *Paper 1*: Attention is all you need sec. 1-5

5.3 Lecture 3

> **Paper to read**
>
> *Paper 1*: How to keep pushing ML accelerator performance? Know your rooflines!

5.4 Lecture 5

> **Paper to read**
>
> *Paper 1*: ENVISION: A 0.26-to-10TOPS/W Subword-Parallel Dynamic-Voltage-Accuracy-Frequency-Scalable Convolutional Neural Network Processor in 28nm FDSOI

6 Questions

6.1 Lecture 1

1. *What is Dennard's law and how is it linked to the evolution of computer architectures over the last 30 years? Describe the different phases we see in this evolution, and the architectural consequences. Illustrate this with examples from processors discussed in class and the papers to be read.*

> **To be answered**

2. *Why do Henessey and Patterson say it is a New Golden Age for computer architectures, and which opportunities should be exploited in this Golden Age, according to them? (See L1_Turing.pdf)*

> **To be answered**

3. *Discuss the different types of processors present in a modern embedded system (like iPhone's, smart glasses, Tesla cars, or…). In what sense do they differ? Why are they all there? How do they exploit area for efficiency? (after L8: How would they exchange data and processing jobs?)*

> **To be answered**

4. *Apply the different concepts from this class to the Apple M1 processor. Which "area for efficiency" techniques do they exploit and why? What is unique about the FireStorm cores. (See also paper L1_Apple.pdf)*

> **To be answered**

5. *What are the reasons for the going to multi-core CPU's, first homogeneous and later heterogeneous? How are the micro-architectures of the CPU's different/similar, and how are they exploited? Does this offer energy efficiency and/or carbon footprint benefits?*

> **To be answered**

6.2 Lecture 2

6. What are ISP's and IPU's? Explain the evolution in their architecture, using example processors to illustrate the trends. What are the main characteristics that distinguish the Google Pixel IPU from a CPU and from the Hexagon processor?

> **To be answered**

7. Explain the workload of embedded rendering tasks and how this leads to new GPU processing architectures beyond CPU's. What are the main characteristics that distinguish GPU's from CPU's? How do mobile GPU's differ from cloud GPU's?

> **To be answered**

8. What is the difference between a fully connected, a convolutional neural network layer and a depthwise separable layer. Which one is considered more hardware efficient, and why?

> **To be answered**

9. What is the attention mechanism in transformers, and what operations does it require in prefill and decode? what are its benefits and downsides compared to convolutional networks. (Also use L2_Attention.pdf)

> **To be answered**

10. (After having studied L3-5) How can certain types of neural network layers be more efficient from an algorithmic point of view and at the same time be more inefficient from a HW point of view?

> **To be answered**

6.3 Lecture 3

11. How can Rooflines be used to assess NPU performance? How do the techniques we discussed in L3-6 impact the rooflines themselves, or the position of a workload in the roofline. (also use paper L3_Rooflines).

> **To be answered**

12. [After L4:] What is the difference between spatial and temporal unrolling of for-loops, and how does it impact the arithmetic intensity? What sub-types of spatial and temporal unrolling can you distinguish?

> **To be answered**

13. Explain the difference between the GeMM execution dataflows of:
    1. Tesla's NPU
    2. An Nvidia TensorCore

> **To be answered**

14. How can spatial unrolling be extended from the datapath to the core level? What sharding opportunities exist and what are their benefits or downsides?

> **To be answered**

15. [After L4:] Given a nested (par)for loop representation (see examples in L4), explain me what the data parallelism and the stationarity is (spatial and temporal unrolling). Also derive the AI.

> **To be answered**

6.4 Lecture 5

16. What data types are relevant for ML computation? What are their main differences? How can this be exploited in an ML processor ans what is the impact on the roofline model? Also use paper L5_MoonsISSCC.

> **To be answered**

17. What different ways are there to build precision scalable MAC units / MAC arrays, and what is the impact on the nested for-loop representation? Use this to discuss the dataflow and utilization consequences (opportunities and challenges) of adapting the MAC precision?.

> **To be answered**

18. Discuss the parallelism, stationarity and sparsity aspects (after L6) of the Envision processor and the Nvidia Tensor Cores, and how these aspects depend on the chosen data type.

> **To be answered**

19. What are the opportunities that come from extreme binary quantization (in the digital, mixed-signal and the in-memory domain? You can use BinarEye as an illustration.

> **To be answered**

20. What is the difference between analog and digital in-memory compute and what are their relative benefits and weaknesses?

> **To be answered**

7 License

7.1 Modification

To contribute to this work or any other one from this project please find more information at the Github repository.
